

Practical 7

```
In [14]: import numpy as np
import random

# =====
# Environment
# =====
class WindyGridWorld:
    def __init__(self):
        self.rows = 7
        self.cols = 10

        # Wind strength
        self.wind = [0, 0, 0, 1, 1, 1, 2, 2, 1, 0]

        self.start = (3, 0)
        self.goal = (3, 7)

        # Actions: 0=Up, 1=Down, 2=Left, 3=Right
        self.actions = [0, 1, 2, 3]

    def step(self, state, action):
        row, col = state

        # Apply action
        if action == 0: row -= 1
        elif action == 1: row += 1
        elif action == 2: col -= 1
        elif action == 3: col += 1

        # Apply wind
        if 0 <= col < self.cols:
            row -= self.wind[col]

        # Boundary conditions
        row = max(0, min(row, self.rows - 1))
        col = max(0, min(col, self.cols - 1))

        next_state = (row, col)
        reward = -1
        done = (next_state == self.goal)

        return next_state, reward, done

# =====
# Epsilon-Greedy Policy
# =====
def epsilon_greedy(Q, state, epsilon):
    if random.random() < epsilon:
        return random.randint(0, 3)
    return np.argmax(Q[state])

# =====
# Q-Learning Algorithm
# =====
def q_learning(env, episodes=500, alpha=0.5, gamma=1.0, epsilon=0.1):

    # Initialize Q-table
    Q = {(i, j): np.zeros(4) for i in range(env.rows) for j in range(env.cols)}

    episode_lengths = []

    for ep in range(episodes):
        state = env.start
        steps = 0
```

```

while True:
    action = epsilon_greedy(Q, state, epsilon)

    next_state, reward, done = env.step(state, action)

    # Q-Learning Update (Off-policy TD)
    Q[state][action] += alpha * (
        reward + gamma * np.max(Q[next_state]) - Q[state][action]
    )

    state = next_state
    steps += 1

    if done:
        break

    episode_lengths.append(steps)

    # ♦ Show Learning progress every 100 episodes
    if (ep + 1) % 100 == 0:
        avg_steps = np.mean(episode_lengths[-100:])
        print(f"Episode {ep+1}, Avg Steps (last 100): {avg_steps:.2f}")

return Q, episode_lengths

# =====
# Extract Policy
# =====
def get_policy(Q, env):
    policy = {}
    for i in range(env.rows):
        for j in range(env.cols):
            policy[(i, j)] = np.argmax(Q[(i, j)])
    return policy

# =====
# Display Policy (Readable)
# =====
def print_policy(policy, env):
    symbols = ['↑', '↓', '←', '→']

    print("\nOptimal Policy:\n")
    for i in range(env.rows):
        row = []
        for j in range(env.cols):
            state = (i, j)
            if state == env.goal:
                row.append(' G ')
            else:
                row.append(f" {symbols[policy[state]]} ")
        print(" ".join(row))

# =====
# Run Training
# =====
env = WindyGridWorld()
Q, episode_lengths = q_learning(env, episodes=1000)

print("\nTraining completed!")
print("Steps in last episode:", episode_lengths[-1])

policy = get_policy(Q, env)
print_policy(policy, env)

```

Episode 100, Avg Steps (last 100): 68.95
Episode 200, Avg Steps (last 100): 19.89
Episode 300, Avg Steps (last 100): 19.89
Episode 400, Avg Steps (last 100): 19.54
Episode 500, Avg Steps (last 100): 19.81
Episode 600, Avg Steps (last 100): 19.48
Episode 700, Avg Steps (last 100): 19.67
Episode 800, Avg Steps (last 100): 19.69
Episode 900, Avg Steps (last 100): 19.75
Episode 1000, Avg Steps (last 100): 19.48

Training completed!
Steps in last episode: 17

Optimal Policy:

↓	→	→	→	→	→	→	→	→	↓
←	↓	→	→	→	→	→	↓	→	↓
→	→	→	→	→	→	↑	↑	→	↓
→	→	→	→	→	→	↑	G	→	↓
→	→	→	→	→	↑	↑	↑	→	↓
→	↓	↑	→	↑	↑	↑	↑	←	↓
↓	↑	↓	↑	↑	↑	↑	↑	↑	←